

Introduction

Most interviewers don't actually care whether you've memorized LeetCode 3302. They care whether you can spot a *shape*. This problem — find the lexicographically smallest sequence of indices from one string that spells another string, give or take one mismatch — belongs to a family that shows up constantly in real systems: spell checkers, fuzzy search, diff tools, even DNA sequence comparison where a single mutation shouldn't disqualify a match. The moment you recognize "match a sequence while tolerating exactly one error," a specific toolkit clicks into place: two pointers, a feasibility check, and one precomputed array that turns an expensive repeated question into a single lookup.

That's what makes this problem worth sitting with. It's not a trick question with a clever one-liner. It's a template for an entire category of "almost matches" problems, and once you've built it once, you'll recognize it everywhere.

Problem Overview

You're given two strings, `word1` and `word2`. You need to choose a strictly increasing sequence of indices from `word1` — as few as possible... actually, the length is fixed at `len(word2)`, since you need to eventually produce `len(word2)` positions. What varies is *which* indices you choose, and among all valid choices, you want the one that is lexicographically smallest when compared as a list of indices (meaning: prefer smaller indices earlier, as early as possible).

The sequence is valid if, when you read the characters at those indices from `word1` in order, the result either: - exactly equals `word2`, or - differs from `word2` in exactly one position (or zero — meaning the "allowed mismatch" is optional, not mandatory).

If no such sequence exists, return an empty list.

The key constraint that makes this interesting: you only get to "spend" one mismatch. Everything else must line up exactly.

Example

Let `word1 = "vbcca"` and `word2 = "abc"`.

- Position 0: `word1[0] = 'v'`, `word2[0] = 'a'` — mismatch. Spend the swap here.

- Position 1: `word1[1] = 'b'` , `word2[1] = 'b'` — free match.
- Position 2: `word1[2] = 'c'` , `word2[2] = 'c'` — free match.

Result: `[0, 1, 2]` . We spent our one allowed mismatch immediately at position 0, because delaying it would have meant either missing the `a` entirely or picking a later, larger index — and larger indices make the sequence lexicographically worse.

A second example: `word1 = "bacdc"` , `word2 = "abc"` .

- Position 0: `'b'` vs `'a'` — mismatch, but check feasibility first (explained below). Not safe yet, skip.
- Position 1: `'a'` vs `'a'` — free match. Take index 1.
- Position 2: `'c'` vs `'b'` — mismatch. Feasibility check passes here. Spend the swap. Take index 2.
- Position 3: `'d'` vs `'c'` — we still need `'c'` , but `word1[3] = 'd'` doesn't match, and we already used our swap. Skip.
- Position 4: `'c'` vs `'c'` — free match. Take index 4.

Result: `[1, 2, 4]` .

Intuition

Picture reading two books with one finger on each page, moving forward in lockstep. Every time the pages agree, take the index for free — there's no reason to hesitate, since a free match never costs you anything and never risks the sequence being too long to finish. Free matches are always correct to take immediately, because delaying a free match only pushes your progress on `word2` later, making the final sequence lexicographically larger.

The interesting decision happens on a mismatch. You have exactly one "get out of jail" pass. Do you spend it now, or hold out for a better spot?

Here's the insight: since you want the *smallest* indices, and you're walking forward, spending the mismatch as early as possible is always at least as good as spending it later — provided it's still *safe* to do so. "Safe" means: after spending it here, can the rest of `word1` still supply every remaining character of `word2` ? If yes, spend it immediately. If no, walking further left in `word1` isn't an option (you can only move forward), so you skip this character and try the next one.

That "can the rest still finish" question needs to be free to ask, because in the worst case you'll ask it at nearly every position. Asking it by scanning ahead each time is what makes the brute-force approach slow. Instead, build a **suffix count array**: walking `word1` backward, precompute, for each starting

index `i`, how many trailing characters of `word2` could still be matched using `word1[i:]`. Once that's built, the feasibility question is just: "does the suffix count at `i+1` cover everything I still need of `word2`?" — a single comparison.

Brute Force Solution

Idea: At every mismatch encountered while walking forward, tentatively spend the swap there, then check — by scanning the rest of `word1` — whether the remaining letters of `word2` can still be completed. If yes, commit; if no, keep looking.

Algorithm: 1. Walk `word1` with a pointer into `word2`. 2. On a match, take it for free. 3. On a mismatch, if the swap hasn't been used, scan ahead from the next index to see if the rest of `word2` can still be finished. 4. If it can, spend the swap and continue; otherwise move past this character.

Advantages: Conceptually simple, easy to reason about correctness.

Disadvantages: The "can it still finish" scan can run at almost every position, and each scan itself takes up to $O(m)$ time (where $m = \text{len}(\text{word2})$), so the whole approach becomes $O(n \cdot m)$ in the worst case.

Time complexity: $O(n \cdot m)$ **Space complexity:** $O(1)$ beyond the output list

Optimal Solution

The fix is to stop re-deriving the same answer over and over. Build the suffix count array once, in $O(n)$ time, and every feasibility check afterward becomes $O(1)$.

Building the suffix array:

Size it `n + 1`, where `n = len(word1)`. The last slot (`suffix[n]`) means "nothing left to match" and is always 0. Walking backward from `i = n - 1` down to `0`:

```
suffix[i] = suffix[i + 1] + 1   if word1[i] == word2[suffix[i + 1]] is reachable
suffix[i] = suffix[i + 1]      otherwise
```

More precisely: `suffix[i]` represents how many characters of `word2`, counted from the end, could still be matched starting from `word1[i:]`. It grows by one exactly when `word1[i]` equals the character of `word2` that would extend the matchable tail.

index in word1 b a c d c

suffix count 1 2 3 3 3

(Reading `word1 = "bacdc"` against `word2 = "abc"` — walking backward, position 4's `c` extends the tail to 1, position 3's `d` doesn't extend it further, position 2's `c` extends it to... the exact bookkeeping is easiest to see in code below.)

The forward walk:

Walk `word1` with a pointer `j` into `word2` and a `swapped` flag.

- If `word1[i] == word2[j]` : take index `i` for free, advance `j` .
- Else if `not swapped` and `suffix[i + 1] >= len(word2) - j - 1` : this position can absorb the mismatch and everything after it can still finish — spend the swap, take index `i` , advance `j` , set `swapped = True` .
- Else: skip this character of `word1` .
- Stop once `j == len(word2)` .

If the walk ends without finishing `word2` , return `[]` .

Python Code

```
def valid_sequence(word1: str, word2: str) -> list[int]:
    n, m = len(word1), len(word2)

    # suffix[i] = how many trailing characters of word2 can still be
    # matched using word1[i:], scanning forward from word1[i].
    suffix = [0] * (n + 1)
    j = m - 1
    for i in range(n - 1, -1, -1):
        suffix[i] = suffix[i + 1]
        if j >= 0 and word1[i] == word2[j]:
            suffix[i] += 1
            j -= 1

    result = []
    j = 0
    swapped = False

    for i in range(n):
        if j == m:
            break
        if word1[i] == word2[j]:
            result.append(i)
            j += 1
        elif not swapped and suffix[i + 1] >= m - j - 1:
```

```

        result.append(i)
        j += 1
        swapped = True

    return result if j == m else []

```

Code Walkthrough

- `suffix[i + 1]` is computed before `suffix[i]` because we walk backward — each position depends only on the one after it.
- `j` during the backward pass tracks how far into `word2`, from the end, we've matched so far. When `word1[i]` matches the next needed character (`word2[j]`), we extend the tail and decrement `j`.
- In the forward walk, `word2 - j - 1` is how many characters of `word2` would still remain *after* using this position to absorb the mismatch. Comparing that against `suffix[i + 1]` (what's achievable from the rest of `word1`) is the whole feasibility check.
- `swapped` ensures we only ever spend the one allowed mismatch once.
- The loop breaks early once `word2` is fully matched — nothing after that matters for the answer.

Dry Run

`word1 = "bacdc"`, `word2 = "abc"` (`n = 5`, `m = 3`).

Backward pass (target sequence to match, right to left: `c`, `b`, `a`):

- `i=4`: `word1[4]='c'` matches `word2[2]='c'` → `suffix[4] = suffix[5] + 1 = 1`, `j` moves to 1.
- `i=3`: `word1[3]='d'`, needs `word2[1]='b'` → no match → `suffix[3] = suffix[4] = 1`.
- `i=2`: `word1[2]='c'`, needs `word2[1]='b'` → no match → `suffix[2] = suffix[3] = 1`.
- `i=1`: `word1[1]='a'`, needs `word2[1]='b'` → no match → `suffix[1] = suffix[2] = 1`.
- `i=0`: `word1[0]='b'`, needs `word2[1]='b'` → match → `suffix[0] = suffix[1] + 1 = 2`, `j` moves to 0.

`suffix = [2, 1, 1, 1, 0]`.

Forward pass:

- `i=0`: `'b'` vs `word2[0]='a'` — mismatch. Check: `suffix[1] = 1 >= 3 - 0 - 1 = 2`? No. Skip.
- `i=1`: `'a'` vs `'a'` — free match. Take 1, `j=1`.

- `i=2 : 'c' vs word2[1]='b'` — mismatch. Check: `suffix[3] = 1 >= 3 - 1 - 1 = 1 ?`
Yes. Spend swap. Take 2, `j=2`, `swapped=True`.
- `i=3 : 'd' vs word2[2]='c'` — mismatch, already swapped. Skip.
- `i=4 : 'c' vs 'c'` — free match. Take 4, `j=3`. Loop ends (`j == m`).

Result: `[1, 2, 4]` — matches the earlier hand-traced example.

Complexity Analysis

- **Time:** $O(n + m)$ — one backward pass to build the suffix array, one forward pass to build the answer. No nested scanning.
- **Space:** $O(n)$ for the suffix array, plus $O(m)$ for the output in the worst case.

This is correct because every feasibility check that used to require an $O(m)$ scan in the brute force now resolves in $O(1)$ via a precomputed value — the total work done across the backward pass is bounded by `n`, and the forward pass is bounded by `n` as well.

Alternative Solutions

A dynamic programming formulation (state = position in `word1`, position in `word2`, swap used or not) also works and is a natural first instinct given the "one allowed error" phrasing. It's correct but unnecessary here — the greedy property (always prefer the earliest safe match) holds because the problem is asking for the lexicographically smallest index sequence, and DP would add $O(n \cdot m)$ states without buying anything the greedy doesn't already get in linear time.

Edge Cases

- **word2 longer than word1** : impossible to match even with the swap — should return `[]`.
The algorithm naturally handles this since `j` will never reach `m`.
- **word1 == word2** : no swap needed at all; every position is a free match.
- **All characters in word1 identical, word2 needs variety**: as shown in the video's quiz (`word1 = "aaaaaa"`, `word2 = "aaabc"`), free matches run out and the swap can't manufacture two missing distinct characters — returns `[]`.
- **Empty word2** : trivially valid with an empty sequence (`[]`), since there's nothing to match — worth confirming against the exact problem constraints, as some variants disallow empty `word2`.

- **Mismatch needed at the very last character:** the suffix check at that point should reduce to comparing against `suffix[n] = 0`, i.e., "nothing left to match after this" — the algorithm handles it without special-casing.

Common Mistakes

1. **Spending the swap on the first mismatch without checking feasibility.** This strands the walk exactly like the naive greedy in the video — it grabs the free `a` and then has no way to recover the `b`.
2. **Rescanning `word1` for every feasibility check instead of precomputing the suffix array**, which silently degrades performance to quadratic on large inputs.
3. **Off-by-one errors in the suffix array size or indexing** — forgetting the extra `n+1` th slot that represents "nothing left to match" breaks the base case.
4. **Allowing the swap to be spent more than once** by forgetting to gate the mismatch branch on the `swapped` flag.
5. **Comparing suffix counts against the wrong remaining length** — using `m - j` instead of `m - j - 1` (since the current position at a mismatch already accounts for one character of `word2`).

Interview Questions

- How would you extend this to allow k mismatches instead of exactly one?
- What's the lexicographically smallest sequence if the problem instead wanted the *largest* set of indices, or the smallest sequence length rather than fixed length?
- Can you do this without extra space, or is $O(n)$ space unavoidable given the suffix array approach?
- How does this pattern relate to the classic "is subsequence" problem, and where does it diverge?

Similar Problems

- **LeetCode 392 (Is Subsequence)** — the zero-mismatch version of this exact problem; a good warm-up before tackling the one-swap variant.
- **LeetCode 792 (Number of Matching Subsequences)** — same subsequence-matching mechanics, different objective.
- **LeetCode 727 (Minimum Window Subsequence)** — another two-pointer-plus-precomputation pattern over subsequences.

- **Edit Distance (LeetCode 72)** — the general "allow k errors" family this problem is a constrained special case of.

Key Takeaways

The pattern is: take every free match immediately, and only spend a limited "exception budget" when a precomputed check proves the rest can still finish. Precompute once, from the back, so that a question you'll ask repeatedly during a forward pass costs $O(1)$ each time instead of $O(m)$. This shape — one error budget, one feasibility gauge — generalizes far beyond this single LeetCode problem.

Watch the Video

For the full build — code written piece by piece, a hand-traced dry run, a predict-before-reveal challenge, and a head-to-head complexity comparison against brute force — watch the full walkthrough here: {LINK}

About the Series

This article is part of **Fun with Learning Technology**, a series that takes one coding problem a day and actually understands it — the intuition, the failure modes, and the reasoning behind the optimal approach — rather than memorizing a solution to pattern-match against.

Call To Action

If this breakdown helped, watch the full video on YouTube and hit subscribe for daily problem walkthroughs. Subscribe to the Substack for the written version delivered straight to your inbox. Drop a comment with your own approach or a problem you'd like covered next, and share this with someone prepping for interviews.

Quick Review

Pattern: swap sequence must be lexicographically smallest with limited edits.

Trigger?

Greedy 'one swap' pattern: when you see 'smallest sequence with ≤ 1 fix allowed', think leftmost-mismatch greedy swap.

Time complexity of the One Swap Greedy solution?

$O(n)$, single left-to-right pass tracking mismatches and one candidate swap.

Space complexity of the One Swap Greedy approach?

$O(1)$ extra (besides output array) — just a few counters/flags, no auxiliary structures.

Trickiest edge case in this problem?

Using the swap too early on a mismatch that would've resolved itself, wasting your one allowed fix and failing the finish-check.

One Swap Greedy vs brute-force try-every-swap: key difference?

Greedy checks feasibility ('can it still finish?') incrementally in one pass; brute force retries all $O(n)$ swaps, costing $O(n^2)$.

Interview follow-up: how do you verify a candidate swap still lets the sequence finish validly?

Continue the scan after swapping; if any further mismatch appears, that swap point was invalid — track validity with a running check, not a restart.